



The Department of Mathematics and Computer Science

[About](#)
[Statistics](#)
[Number Theory](#)
[Java](#)
[Data Structures](#)
[Cornerstones](#)
[Calculus](#)

The Two's Complement

Sign-and-Magnitude Method

Consider the problem of representing both positive AND negative integers over a given range in terms of only ones and zeroes. A straight-forward approach would be to deal with the sign and the magnitude (or, the absolute value) separately. For example, suppose we have 8 bits with which to work. The first bit could represent the sign of our number ("0" for a positive number, "1" for a negative number), while the other 7 bits could be a binary representation of the magnitude of the number. With only 7 bits to express it, the magnitude must range from zero to 127 (i.e., 0000000 to 1111111). Consider the following examples:

Value	Representation in 8 Bits	Value	Representation in 8 Bits
3	00000011	-3	10000011
14	00001110	-14	10001110
82	01010010	-82	11010010
127	01111111	-127	11111111

There are problems with this approach, however. Consider the case of zero. Should it be encoded as 00000000 or 10000000?

As another, more significant issue, consider how one is forced to add values. Our process depends on our context. If both numbers are positive, one can simply use normal binary addition:

```

  5      0 0000101
+3      0 0000011
-----
  8      0 0001000 (which represents 8)

```

But if there is a mixture of signs, we need to recognize this requires subtraction, as otherwise we get an incorrect result:

```

  5      00000101
+(-3)    10000011
-----
  2      10001000 (which represents -8)

```

In the interests of efficiency, it would be nice if the process we use internally to add two numbers didn't depend on their values! Enter the two's complement...

The Two's Complement Method

In the (8-bit) two's complement representation of a integer value between -127 and 127, positive integers are represented in the normal binary manner (with a natural leading zero as the largest number is 127). Representations for negative numbers are obtained from the representations for positive numbers in the following way:

	Example 1: (value = -41)	Example 2: (value = -44)
1. Starting from the right, find the first '1'	00101001	00101100
2. Invert all of the bits to the left of that one	11010111	11010100

Here are some more examples:

Value	Two's Complement Representation	Value	Two's Complement Representation
3	00000011	-3	11111101
14	00001110	-14	11110010
82	01010010	-82	10101110
127	01111111	-127	10000001

Amazingly, representing negative numbers in this way allows us to compute the sum of two numbers, regardless of their signs, in the SAME way -- via normal binary addition!

```

  5      00000101
+3      00000011
-----
  8      00001000 (which represents 8)

```

```

  5      00000101
+(-3)    11111101
-----
  2      00000010 (which represents 2)

```

If that wasn't enough, note that we get the added bonuses of now having only one representation for zero (00000000), and we can actually include one more number in our range (-128 = 10000000), without causing any problems. Pretty cool, eh? Makes one wonder how they came up with this process for negation, doesn't it?

Think about it though... The negative of a number is defined as the value that can be added to the original number to produce a zero. For example, consider -84 . This is defined to be the number that can be added to 84 to produce a sum of zero. We know that 84 written in binary is given by 01010010 after adding the leading zero so that we have 8 bits total. Let $d_7d_6d_5d_4d_3d_2d_1d_0$ be the binary expansion of -84 . Then we require that

$$\begin{array}{rcccccccc}
 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 0 \\
 + & d_7 & d_6 & d_5 & d_4 & d_3 & d_2 & d_1 & d_0 \\
 \hline
 \dots & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0
 \end{array}$$

By duplicating the right-most **1** (i.e., letting $d_2 = 1$), and all of the zeros to the right of the right-most **1** (i.e., $d_1 = d_0 = 0$), we assure sums of zeros there. Most of these will be of the form $0 + 0 = 0$, while the sum involving the right-most **1** will produce **0** and a carried **1** (recall in binary, $1 + 1 = 10$).

$$\begin{array}{rcccccccc}
 & & & & & 1 & & & \\
 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 0 \\
 + & d_7 & d_6 & d_5 & d_4 & d_3 & 1 & 0 & 0 \\
 \hline
 & & & & & & 0 & 0 & 0
 \end{array}$$

Then, by inverting all of the bits to the left of the right-most **1** (i.e., d_2 through d_7), we assure each of these columns sum to **1** before considering any "carries". Now, throw in the carried **1** produced from the right-most **1** and its duplicate, and one produces a **0** and another carried **1** in each column working to the left. In essence, the carried **1** simply propagates down the line -- until it disappears when we run out of bits (remember, we restricted ourselves to 8 bits).

$$\begin{array}{rcccccccc}
 & 1 & 1 & 1 & 1 & 1 & & & \\
 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 0 \\
 + & 1 & 0 & 1 & 0 & 1 & 1 & 0 & 0 \\
 \hline
 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0
 \end{array}$$

The result is a bit string consisting of 8 zeros, which is the bit string for zero -- which was exactly what was desired! Cool, isn't it?